

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

АЛГОРИТМЫ

Отчёт по практической работе по дисциплине «Корпоративные
информационные системы»

Студент гр.594-2

_____ А. А. Танцеров

“ _____ ” _____ 2026 г.

Руководитель старший преподаватель
кафедры ЭМИС

_____ А. А. Матолыгин

“ _____ ” _____ 2026 г.

Томск 2026

Практическая работа №0

«Алгоритмы»

Цель работы: изучить базовые элементы синтаксиса языка программирования Java, освоить основные конструкции языка и получить практические навыки написания, компиляции и запуска простых программ в среде разработки.

Java – это высокоуровневый объектно-ориентированный язык программирования, разработанный компанией Sun Microsystems в середине 1990-х годов под руководством Джеймса Гослинга. Работа над языком началась в 1991 году в рамках проекта Green, изначально ориентированного на создание программного обеспечения для бытовой электроники, однако впоследствии разработчики переориентировались на интернет-приложения, что во многом определило дальнейший успех языка. Официальный релиз Java состоялся в 1995 году, и с тех пор язык прошёл долгий путь развития, оставаясь одним из самых востребованных инструментов программирования в мире.

Ключевой особенностью Java является принцип «напиши однажды – выполняй везде» (Write Once, Run Anywhere). Это достигается за счёт использования виртуальной машины Java (Java Virtual Machine, JVM), которая выступает промежуточным звеном между исходным кодом программы и операционной системой компьютера. Программа на Java компилируется не напрямую в машинный код конкретной платформы, а в промежуточный байт-код, который затем интерпретируется или компилируется «на лету» виртуальной машиной, установленной на целевом устройстве. Благодаря этому одна и та же скомпилированная программа может без изменений выполняться на компьютерах с разными операционными системами – Windows, Linux, macOS – при условии наличия соответствующей JVM.

Java является строго типизированным и объектно-ориентированным языком программирования. Это означает, что практически всё в программе

представлено в виде объектов и классов, а типы данных переменных должны быть явно объявлены и не могут произвольно изменяться в процессе выполнения программы. Объектно-ориентированный подход в Java базируется на четырёх фундаментальных принципах: инкапсуляции, наследовании, полиморфизме и абстракции. Инкапсуляция позволяет скрывать внутреннюю реализацию класса и предоставлять доступ к данным только через определённые методы. Наследование даёт возможность создавать новые классы на основе уже существующих, расширяя или изменяя их функциональность. Полиморфизм позволяет объектам разных классов обрабатывать одинаковые вызовы методов по-разному, а абстракция помогает выделять существенные характеристики объектов, скрывая избыточные детали реализации.

Синтаксис Java во многом напоминает синтаксис языков C и C++, что делает переход на Java относительно простым для программистов, знакомых с этими языками. Вместе с тем разработчики Java сознательно отказались от некоторых сложных и потенциально опасных конструкций, таких как прямая работа с указателями и множественное наследование классов, заменив их более безопасными механизмами – например, интерфейсами. Это значительно снижает вероятность возникновения ошибок, связанных с утечками памяти и некорректным доступом к данным. Управление памятью в Java осуществляется автоматически с помощью механизма сборки мусора (garbage collector), который отслеживает объекты, более не используемые программой, и освобождает занимаемую ими память без непосредственного участия разработчика.

Java обладает развитой и хорошо продуманной системой типов данных, разделяемых на примитивные и ссылочные. К примитивным типам относятся числовые типы (byte, short, int, long, float, double), логический тип boolean и символьный тип char. Ссылочные типы включают классы, интерфейсы и массивы. Важной особенностью языка является наличие так называемых обёрточных классов (wrapper classes), которые позволяют представлять

примитивные значения в виде объектов, что бывает необходимо при работе с коллекциями и обобщёнными типами (generics).

Для ознакомления с базовыми конструкциями языка Java реализована консольная программа для работы с массивом целых чисел. Пользователь вводит положительные числа, затем может вывести массив в прямом и обратном порядке, отсортировать его по возрастанию или по убыванию. Управление выполняется через текстовое меню.

В программе использован класс «ArrayList» из пакета java.util. Он выбран вместо обычного массива, потому что заранее неизвестно, сколько чисел введёт пользователь, а «ArrayList» позволяет добавлять элементы без заданного размера. Для чтения данных с клавиатуры использован класс Scanner. Он объявлен как static final, поэтому создаётся один раз на весь класс и не пересоздаётся при каждом вызове метода ввода.

Список хранится в закрытом поле lst. В классе три конструктора: без параметров, с передачей списка и копирующий. Везде, где список передаётся в класс или возвращается из него (конструктор, setLst(), getLst()), создаётся его копия, поэтому изменение исходного списка снаружи не затрагивает данные внутри объекта.

Метод enterDigits() запрашивает числа, пока пользователь не введёт «-1». Отрицательные числа и текст отклоняются с сообщением об ошибке. Если введено меньше двух чисел, ввод запрашивается заново.

Сортировка выполняется методом Collections.sort(). Метод sortAsc() сортирует список по возрастанию, а sortDesc() сначала сортирует его по возрастанию, затем разворачивает методом Collections.reverse(). Вывод в прямом и обратном порядке выполняют методы coutArrayForward() и coutArrayBack(). Проверка, что в списке есть минимум два числа, вынесена в закрытый метод hasEnoughData().

Метод menu() выводит меню в цикле do-while, пока не выбран пункт 6. Выбранный пункт обрабатывается оператором switch. Если вместо числа введён текст, программа выводит сообщение и показывает меню снова.

Код класса Zero.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Scanner;

class Zero {
    private ArrayList<Integer> lst;
    private static final Scanner scanner = new
Scanner(System.in);

    public Zero() {
        this.lst = new ArrayList<>();
    }

    public Zero(ArrayList<Integer> lst) {
        this.lst = new ArrayList<>(lst); // Если кто-то удалит
из старого, то в новом ничего не изменится
    }

    public Zero(Zero other) {
        this.lst = new ArrayList<>(other.lst);
    }

    public ArrayList<Integer> getLst() {
        return new ArrayList<>(lst);
    }

    public void setLst(ArrayList<Integer> lst) {
        this.lst = new ArrayList<>(lst);
    }

    public void enterDigits() {
        ArrayList<Integer> result = new ArrayList<>();

        do {
            System.out.println("Вводите положительные числа
(минимум 2).");
            System.out.println("Чтобы закончить ввод, впишите -
1.");
            boolean flag = true;
            while (flag) {
                System.out.print("-> ");
                if (scanner.hasNextInt()) {
                    int num = scanner.nextInt();
                    if (num == -1) {
                        flag = false;
                    } else if (num < 0) {
                        System.out.println("Число должно быть
положительным. Попробуйте ещё раз.");
                    } else {
                        result.add(num);
                    }
                }
            }
        } while (true);
    }
}
```

```

        } else {
            scanner.next();
            System.out.println("Это не число, попробуй
ещё раз.");
        }
    }

    if (result.size() < 2) {
        System.out.println("\nОшибка: Вы ввели менее 2-х
чисел. Давайте начнем заново.\n");
    }

    } while (result.size() < 2);

    this.lst = result;
}

public void sortAsc() {
    Collections.sort(this.lst);
}

public void sortDesc() {
    sortAsc();
    Collections.reverse(this.lst);
}

private boolean hasEnoughData() {
    return this.lst.size() > 1;
}

public void menu() {
    int choice = 0;

    do {
        System.out.print("""

        Введите число (1 - 5):
        1. Показать массив в прямом порядке;
        2. Показать массив в противоположном
порядке;
        3. Ввести числа;
        4. Сортировка по убыванию;
        5. Сортировка по возрастанию;
        6. Выход.
        Выбор: """);

        if (scanner.hasNextInt()) {
            choice = scanner.nextInt();
            switch (choice) {
                case 1:
                    if (hasEnoughData()) {
                        coutArrayForward();
                    } else {

```

```

        System.out.println("Недостаточно
данных. Сначала заполните массив (минимум 2 числа).");
    }
    break;
case 2:
    if (hasEnoughData()) {
        coutArrayBack();
    } else {
        System.out.println("Недостаточно
данных. Сначала заполните массив (минимум 2 числа).");
    }
    break;
case 3:
    enterDigits();
    break;
case 4:
    if (hasEnoughData()) {
        sortDesc();
        System.out.println("Массив
отсортирован по убыванию.");
    } else {
        System.out.println("Недостаточно
данных. Сначала заполните массив (минимум 2 числа).");
    }
    break;
case 5:
    if (hasEnoughData()) {
        sortAsc();
        System.out.println("Массив
отсортирован по возрастанию.");
    } else {
        System.out.println("Недостаточно
данных. Сначала заполните массив (минимум 2 числа).");
    }
    break;
case 6:
    System.out.println("Завершение
работы.");
    break;
default:
    System.out.println("Нет такого
пункта.");
    }
} else {
    scanner.next();
    System.out.println("Введите корректное число.");
}
} while (choice != 6);
}

public void coutArrayForward() {
    for (int num : this.lst) {
        System.out.print(num + " ");
    }
}

```

```

    }
    System.out.println();
}

public void coutArrayBack() {
    for (int i = this.lst.size() - 1; i >= 0; i--) {
        System.out.print(this.lst.get(i) + " ");
    }
    System.out.println();
}
}

```

Далее создан файл «Main.java», являющийся входной точкой в программу. В нём создаётся объект класса Zero и вызывается метод menu(). Код класса Main.

```

public class Main {
    public static void main(String[] args) {
        Zero lab = new Zero();
        lab.menu();
    }
}

```

Работа программы проверена на нескольких сценариях. Сначала проверена реакция на пустой массив: при запуске выбран пункт 1, затем выполнен выход через пункт 6. Результат представлен на рисунке 1.

```

Введите число (1 - 5):
1. Показать массив в прямом порядке;
2. Показать массив в противоположном порядке;
3. Ввести числа;
4. Сортировка по убыванию;
5. Сортировка по возрастанию;
6. Выход.
Выбор:1
Недостаточно данных. Сначала заполните массив (минимум 2 числа).

Введите число (1 - 5):
1. Показать массив в прямом порядке;
2. Показать массив в противоположном порядке;
3. Ввести числа;
4. Сортировка по убыванию;
5. Сортировка по возрастанию;
6. Выход.
Выбор:6
Завершение работы.

```

Рисунок 1 – Меню и реакция на пустой массив

Далее выбран пункт 3 и введены данные с ошибками: текст «abc» и отрицательное число -4. Программа вывела сообщения об ошибках и продолжила ввод. Числа 5, 3, 8, 2, 9 приняты, ввод завершён числом -1. Результат представлен на рисунке 2.

```
Выбор:3
Вводите положительные числа (минимум 2).
Чтобы закончить ввод, впишите -1.
-> abc
Это не число, попробуйте ещё раз.
-> -4
Число должно быть положительным. Попробуйте ещё раз.
-> 5
-> 3
-> 8
-> 2
-> 9
-> -1
```

Рисунок 2 – Ввод чисел с ошибочными значениями

Затем проверен ввод менее двух чисел: введено одно число 4 и -1. Программа вывела ошибку и запросила ввод заново, после чего введены числа 7 и 6. Результат представлен на рисунке 3.

После ввода чисел 5, 3, 8, 2, 9 выбраны пункты 1 и 2 для вывода массива в прямом и обратном порядке. Результат представлен на рисунке 4.

```
Выбор:3
Вводите положительные числа (минимум 2).
Чтобы закончить ввод, впишите -1.
-> 4
-> -1

Ошибка: Вы ввели менее 2-х чисел. Давайте начнем заново.

Вводите положительные числа (минимум 2).
Чтобы закончить ввод, впишите -1.
-> 7
-> 6
-> -1
```

Рисунок 3 – Ввод менее двух чисел

```
Выбор:1
5 3 8 2 9

Введите число (1 - 5):
1. Показать массив в прямом порядке;
2. Показать массив в противоположном порядке;
3. Ввести числа;
4. Сортировка по убыванию;
5. Сортировка по возрастанию;
6. Выход.
Выбор:2
9 2 8 3 5
```

Рисунок 4 – Вывод массива в прямом и обратном порядке

Для того же набора чисел выбрана сортировка по возрастанию (пункт 5), затем массив выведен пунктом 1. Результат представлен на рисунке 5.

```
Выбор:5
Массив отсортирован по возрастанию.

Введите число (1 - 5):
1. Показать массив в прямом порядке;
2. Показать массив в противоположном порядке;
3. Ввести числа;
4. Сортировка по убыванию;
5. Сортировка по возрастанию;
6. Выход.
Выбор:1
2 3 5 8 9
```

Рисунок 5 – Сортировка по возрастанию

После этого выбрана сортировка по убыванию (пункт 4), и массив снова выведен пунктом 1. Результат представлен на рисунке 6.

```
Выбор:4
Массив отсортирован по убыванию.

Введите число (1 - 5):
1. Показать массив в прямом порядке;
2. Показать массив в противоположном порядке;
3. Ввести числа;
4. Сортировка по убыванию;
5. Сортировка по возрастанию;
6. Выход.
Выбор:1
9 8 5 3 2
```

Рисунок 6 – Сортировка по убыванию

Вывод: изучены базовые элементы синтаксиса языка программирования Java, освоены основные конструкции языка (классы, конструкторы, циклы, условные операторы, switch) и работа с коллекцией ArrayList.